CS1005: Digital Design & Microprocessors

Dr. Vegesna S M Srinivasavarma,
Assistant Professor,
Computer Science & Engineering,
Shiv Nadar University Chennai.

Overview

- Courses includes both theory and practice.
- Theory contains **5 Units**. Each unit will be covered in **9 lecture hours**
- **3** Lecture hours and **1** tutorial hour per week (Total: **4 Hrs**/week)

Syllabus Overview:

Unit-1: INTRODUCTION TO DIGITAL DESIGN

Unit-2: BOOLEAN ALGEBRA AND MINIMIZATION TECHNIQUES

Unit-3: COMBINATIONAL CIRCUIT DESIGN

Unit-4: SEQUENTIAL CIRCUIT DESIGN

Unit-5: MICROPROCESSORS (Intel 8086)

Unit-1: INTRODUCTION TO DIGITAL DESIGN

Topics: Number systems: Decimal, Binary, Octal and Hexadecimal - Conversion from one system to another - Floating point representation of numbers - Arithmetic operation:1's complement, 2's complement - Introduction to Digital Circuits - Advantages and Disadvantages of Digital circuits over Analog circuits - Logic gates -Truth tables

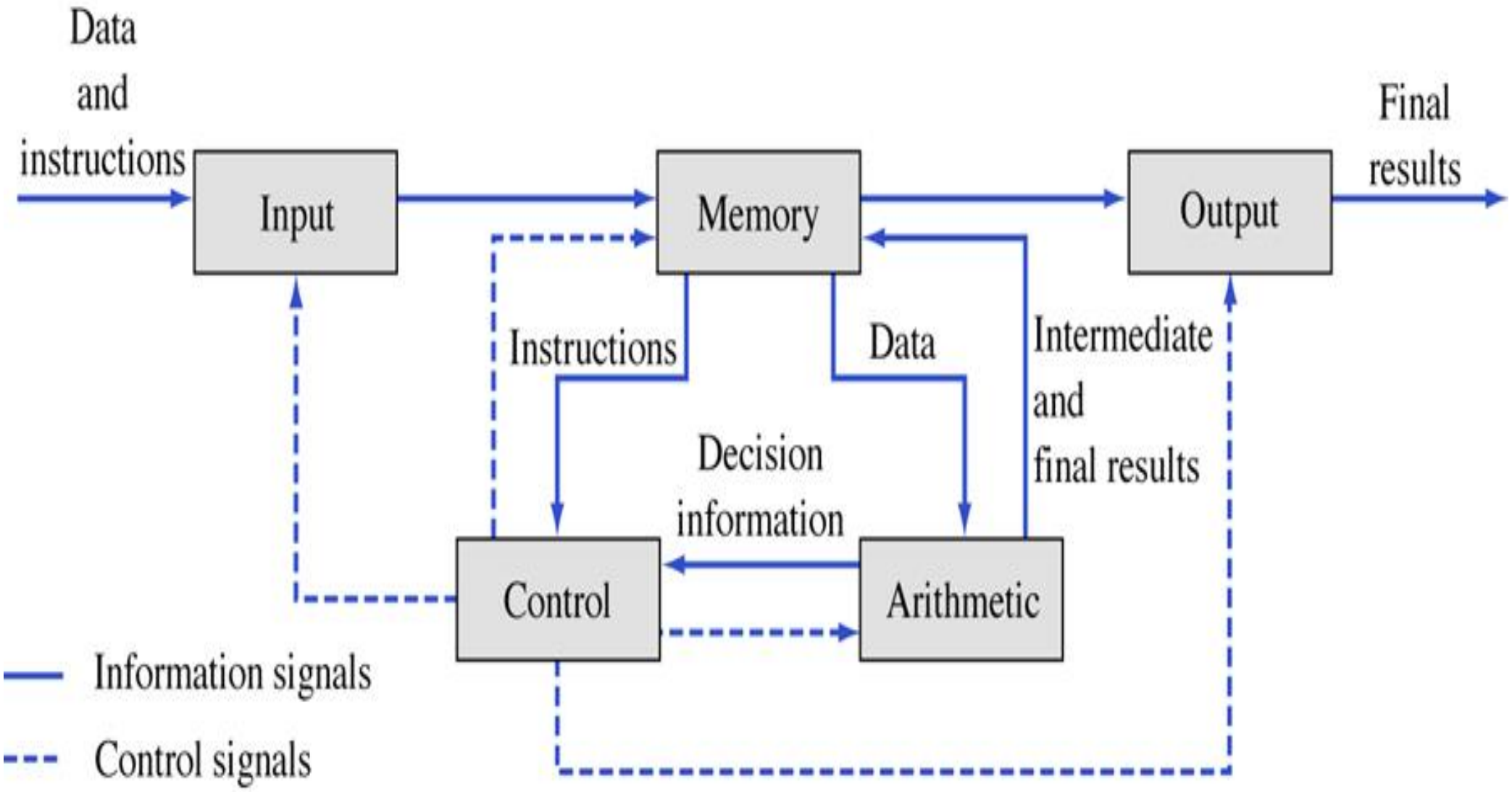
Digital systems

- Digital systems operate on discrete elements of information
  - Numbers (e.g., pocket calculator) -> "digits" -> "digital"
  - Letters (e.g., word processor)
  - Pictures (e.g., digital cameras)
- Some systems use continuous information
  - Examples?
    - » Older thermostats
    - » Film cameras
    - » Slide ruler
- Today:
  - How are numbers represented in digital systems?

Digital Computer Systems

- Digital systems consider *discrete* amounts of data.
- Examples
  - 26 letters in the alphabet
  - 10 decimal digits
- Larger quantities can be built from discrete values:
  - Words made of letters
  - Numbers made of decimal digits (e.g. 239875.32)
- Computers operate on *binary* values (0 and 1)
- Easy to represent binary values electrically
  - Voltages and currents.
  - Can be implemented using circuits
  - Create the building blocks of modern computers

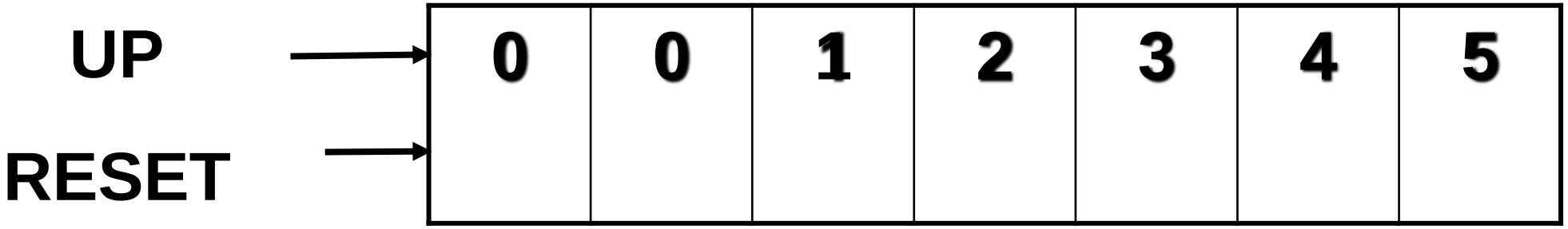
Basic Organization of a Digital Computer



Types of Systems

- With no state present
  - Combinational logic system
  - Output = Function (Input)
- With state present
  - State updated at discrete times (e.g. once per clock tick)

Example : Digital Counter



Inputs : Count Up, Reset

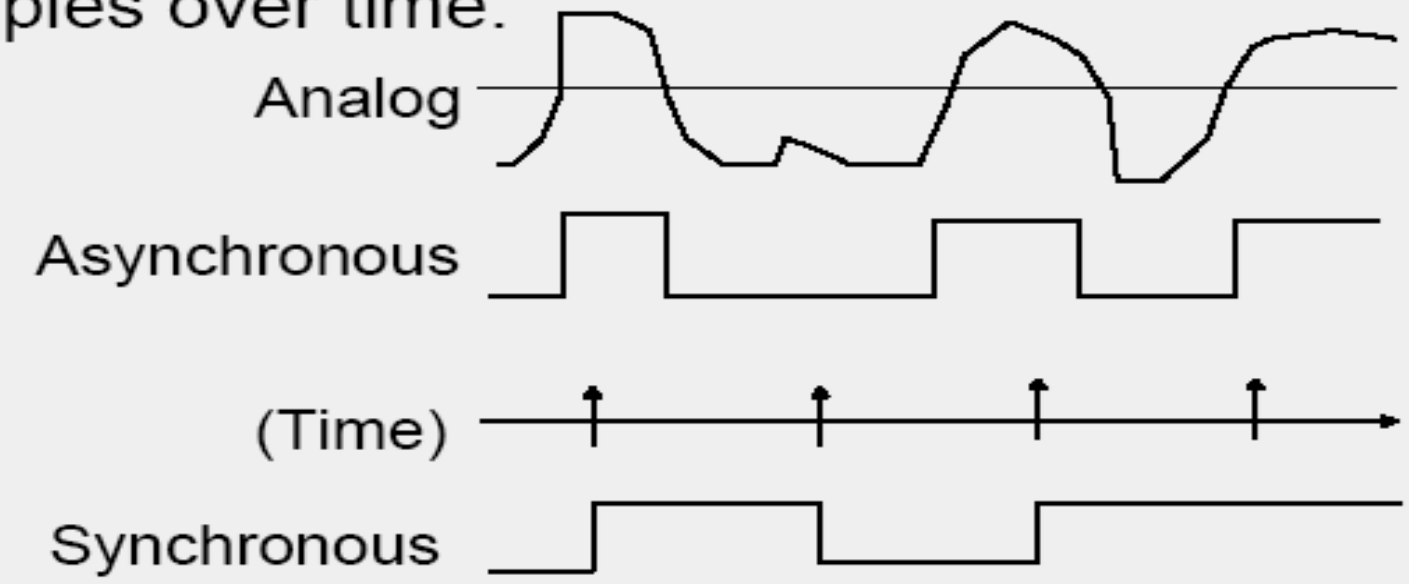
Outputs: Visual Display

State: “Value of stored digits

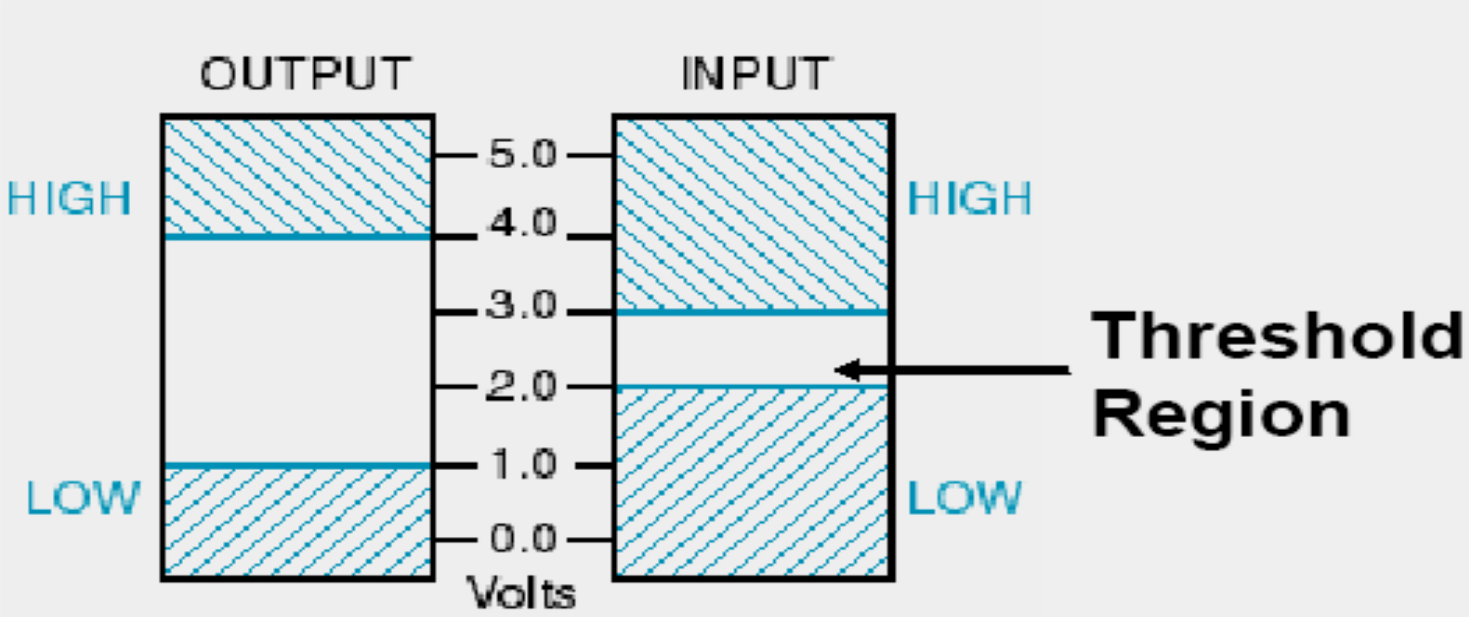
Is this system synchronous or asynchronous?

Signals

- Information variables mapped to physical quantities
- In digital systems, the quantities take on *discrete* values
  - Two-level, or *binary,* values are the most prevalent values in digital systems
  - Binary values are represented abstractly by digits 0 and 1
- Signal examples over time:



Physical Signal Example - Voltage



- **Other physical signals representing 1 and 0**
  - **CPU** — **Voltage**
  - **Disk** — **Magnetic field direction**
  - **CD** — **Surface pits / light**
  - **Dynamic RAM** — **Charge**

Number Systems

- **Decimal Numbers**
  - What does 5,634 represent?
  - Expanding 5,634:

$$
\begin{aligned}
5 \times 10^3 &= 5{,}000 \\
+\ 6 \times 10^2 &= 600 \\
+\ 3 \times 10^1 &= 30 \\
+\ 4 \times 10^0 &= 4 \quad \rightarrow 5{,}634
\end{aligned}
$$

  - What is "10" called in the above expansion?

    The radix.
  - What is this type of number system called?

    Decimal.
  - What are the digits for decimal numbers?

    0, 1, 2, 3, 4, 5, 6, 7, 8, 9.
  - What are the digits for radix-r numbers?

    0, 1, …, r-1.

Understanding Decimal Numbers

- Decimal numbers are made of decimal digits: (0,1,2,3,4,5,6,7,8,9)
- But how many items does a decimal number represent?
  - $8653 = 8x10^3 + 6x10^2 + 5x10^1 + 3x10^0$
- What about fractions?
  - $97654.35 = 9x10^4 + 7x10^3 + 6x10^2 + 5x10^1 + 4x10^0 + 3x10^{-1} + 5x10^{-2}$
  - In formal notation -> $(97654.35)_{10}$

Understanding Octal Numbers

- Octal numbers are made of octal digits: (0,1,2,3,4,5,6,7)
- How many items does an octal number represent?
  - $(4536)_8 = 4x8^3 + 5x8^2 + 3x8^1 + 6x8^0 = (1362)_{10}$
- What about fractions?
  - $(465.27)_8 = 4x8^2 + 6x8^1 + 5x8^0 + 2x8^{-1} + 7x8^{-2}$
- Octal numbers don't use digits 8 or 9
- Who would use octal number, anyway?

Understanding Binary Numbers

- Binary numbers are made of <u>b</u>inary dig<u>it</u>s (bits):
  - 0 and 1
- How many items does an binary number represent?
  - $(1011)_2 = 1x2^3 + 0x2^2 + 1x2^1 + 1x2^0 = (11)_{10}$
- What about fractions?
  - $(110.10)_2 = 1x2^2 + 1x2^1 + 0x2^0 + 1x2^{-1} + 0x2^{-2}$
- Groups of eight bits are called a *byte*
  - $(11001001)_2$
- Groups of four bits are called a *nibble*.
  - $(1101)_2$

Why Use Binary Numbers?

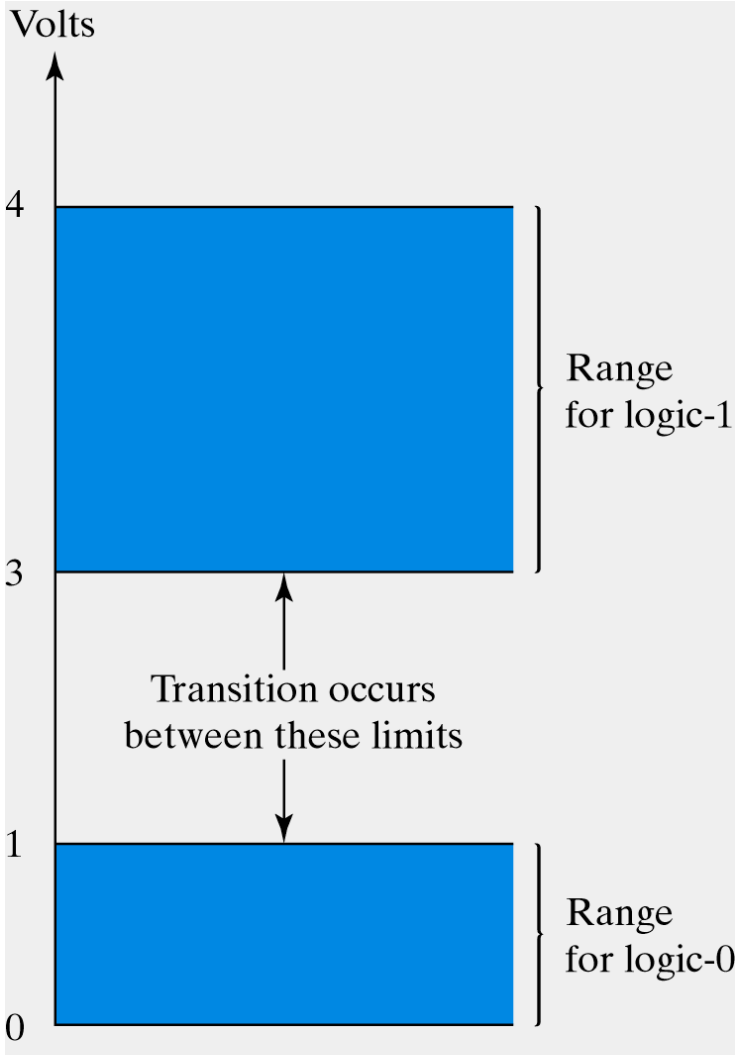
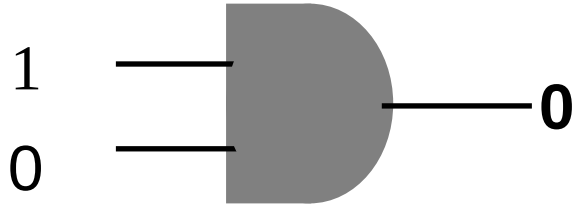


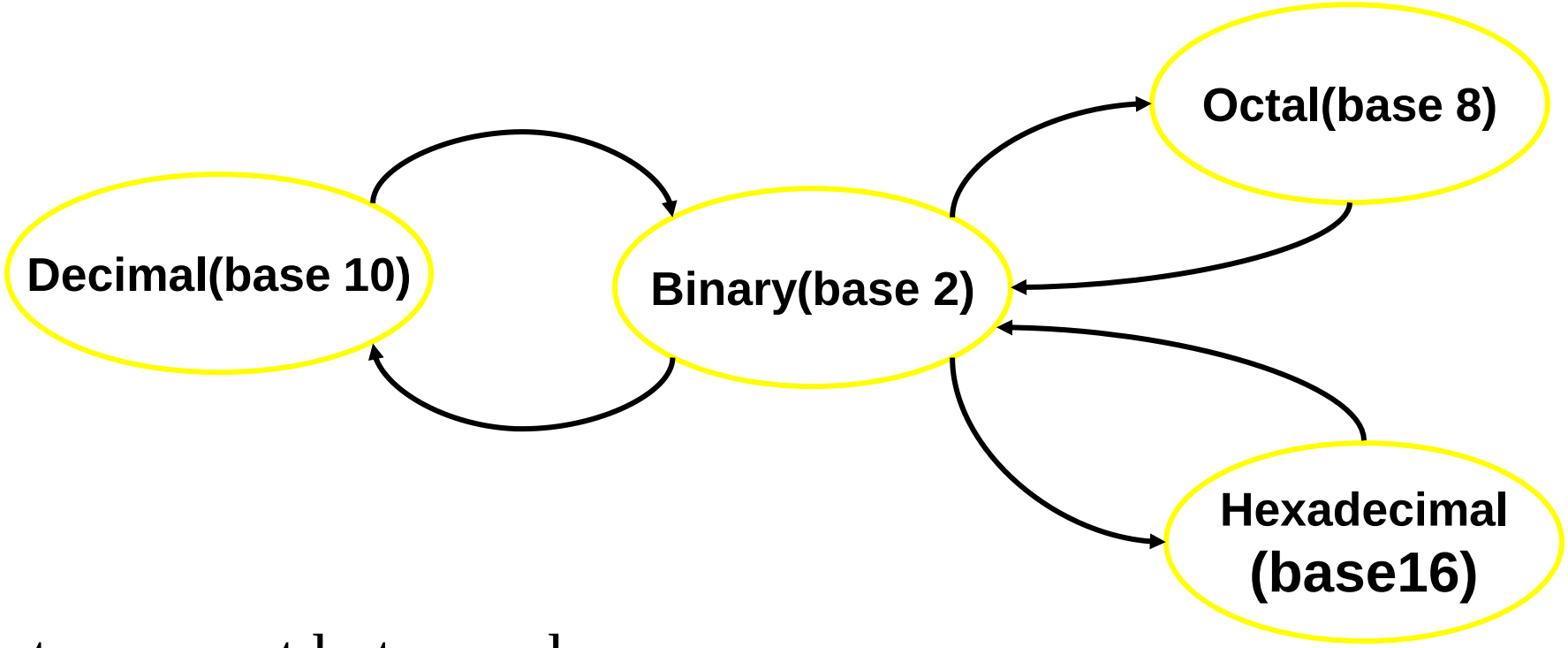
Fig. 1-3 Example of binary signals

- Easy to represent 0 and 1 using electrical values.
- Possible to tolerate noise.
- Easy to transmit data
- Easy to build binary circuits.

AND Gate



Conversion Between Number Bases



- Learn to convert between bases.
- Already demonstrated how to convert from binary to decimal.

General Base Conversion

- **Number Representation**

Given a number of radix r of

"n" <u>integer</u> digits $a_{n-1},\ldots,a_0$

and n

"m" <u>fractional</u> digits $a_{-1},\ldots,a_{-m}$

written as:

$$a_{n-1}\ a_{n-2}\ a_{n-3}\ \ldots\ a_2\ a_1\ a_0\ .\ a_{-1}\ a_{-2}\ \ldots\ a_{-m}$$

has value:

$$(\text{Number})_r = \left(\sum_{i=0}^{i=n-1} a_i \cdot r^i\right) + \left(\sum_{j=-m}^{j=-1} a_j \cdot r^j\right)$$

(Integer Portion) + (Fraction Portion)

Commonly Occurring Bases

Name	Radix	Digits
Binary	2	0,1
Octal	8	0,1,2,3,4,5,6,7
Decimal	10	0,1,2,3,4,5,6,7,8,9
Hexadecimal	16	0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F
		(= 0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15)

Converting Binary to Decimal

- To Convert to decimal, use decimal arithmetic to sum the weighted powers of two:
- Converting 11010_2 to N_{10}:

$$N_{10} = 1 \text{ x } 2^4 \text{ x } 1\text{x } 2^3 + 0 \text{ x } 2^2 + 2^1 + 0 + 2^0$$

$$= 26$$

Converting Decimal to Binary

- **Method 1 (Method 2 – repeated division – next slide)**
  - **Subtract the largest power of 2 that gives a positive result and record the power.**
  - **Repeat, subtracting from the prior result, until the remainder is zero.**
  - **Place 1's in the positions in the binary result corresponding to the powers recorded; in all other positions place 0's.**
- **Example: $625_{10} \rightarrow 1001110001_2$**
  - 625 – 512 = 113 → 9
  - 113 – 64 = 49 → 6
  - 49 – 32 = 17 → 5
  - 17 – 16 = 1 → 4
  - 1 – 1 = 0 → 1
  - Place 1's in the the positions recorded and 0's elsewhere
- **Converting binary to decimal:** sum weighted powers of 2 using decimal arithmetic, e.g., 512 + 64 + 32 + 16 + 1 = 625

Conversion Between Bases

- **Convert the Integral Part**
  - **Repeatedly divide the number by the radix you want to convert to and save the remainders. The new radix digits are the remainders in reverse order of computation.**

    Why does this work?

    This works because, the remainder left in the division is always is the coefficient of the radix's exponent.

    - **If the new radix is > 10, then convert all remainders > 10 to digits A, B, …**
- **Convert the Fractional Part**
  - **Repeatedly multiply the fraction by the radix and save the integer digits that result. The new radix fraction digits are the integer numbers in *computed order.***

    Why does this work?

    To convert fractional part, it should be divided by reciprocal of radix, which is same as multiplying with radix.

    - **If the new radix is > 10, then convert all integer numbers > 10 to digits A, B, …**
- **Join together with the radix point**

Example: Convert 46.6875_{10} To Base 2

- **Convert 46 to Base 2**

  46/2 = 23 remainder = 0
  23/2 = 11 remainder = 1
  11/2 = 5 remainder = 1
  5/2 = 2 remainder = 1
  2/2 = 1 remainder = 0
  1/2 = 0 remainder = 1
  Read off in reverse order: 101110_2

- **Convert 0.6875 to Base 2:**

  0.6875 * 2 = 1.3750 int = 1
  0.3750 * 2 = 0.7500 int = 0
  0.7500 * 2 = 1.5000 int = 1
  0.5000 * 2 = 1.0000 int = 1
  0.0000
  Read off in forward order: 0.1011_2

- **Join together with the radix point: 1011110.1011_2**

Converting Among Octal, Hexadecimal, Binary

- **Octal (Hexadecimal) to Binary:**
  - **Restate the octal (hexadecimal) as three (four) binary digits, starting at radix point and going both ways**
- **Binary to Octal (Hexadecimal):**
  - **Group the binary digits into three (four) bit groups starting at the radix point and going both ways, padding with zeros as needed in the fractional part**
  - **Convert each group of three (four) bits to an octal (hexadecimal) digit**
- **Example: Octal to Binary to Hexadecimal**

$$6\ \ 3\ \ 5\ .\ 1\ \ 7\ \ 7_8$$

$$= 110|011|101\ .\ 001|111|111_2$$

$$= 1|1001|1101\ .\ 0011|1111|1(000)_2 \text{ (regrouping)}$$

$$= 1\ \ 9\ \ D\ .\ 3\ \ F\ \ 8_{16} \text{ (converting)}$$

Number of Bits Required

- Given M elements to be represented by a binary code, the minimum number of bits, *n*, needed satisfies the following relationships:
  - $2^n >= M > 2^{n-1}$
  - $n = \text{ceil}(\log_2 M)$ where ceil(x) is the smallest integer greater than or equal to x
- Example: How many bits are <u>required</u> to represent <u>decimal digits</u> with a binary code?
  - M = 10 → n = 4

Number of Elements Represented

- Given *n* digits in radix *r*, there are r^n distinct elements that can be represented.
- But, can represent m elements, $m < r^n$
- Examples:
  - Can represent 4 elements in radix r = 2 with n = 2 digits: (00, 01, 10, 11)
  - Can represent 4 elements in radix r = 2 with n = 4 digits: (0001, 0010, 0100, 1000)
    - This code is called a "one hot" code

Decimal, BCD, Xcess-3, Binary, Octal, Hexa-Decimal, and Gray Codes:

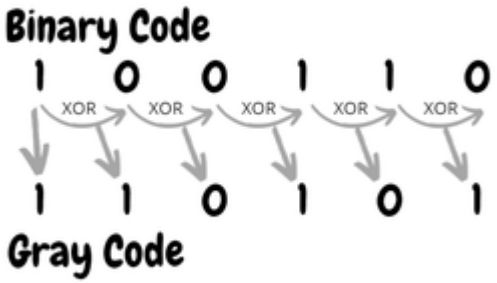
Decimal	BCD	Xcess-3	Binary	Octal	Hexa-Decimal	Gray
0	0000	0011	0000	00	0	0000
1	0001	0100	0001	01	1	0001
2	0010	0101	0010	02	2	0011
3	0011	0110	0011	03	3	0010
4	0100	0111	0100	04	4	0110
5	0101	1000	0101	05	5	0111
6	0110	1001	0110	06	6	0101
7	0111	1010	0111	07	7	0100
8	1000	1011	1000	10	8	1100
9	1001	1100	1001	11	9	1101
10			1010	12	A	1111
11			1011	13	B	1110
12			1100	14	C	1010
13			1101	15	D	1011
14			1110	16	E	1001
15			1111	17	F	1000

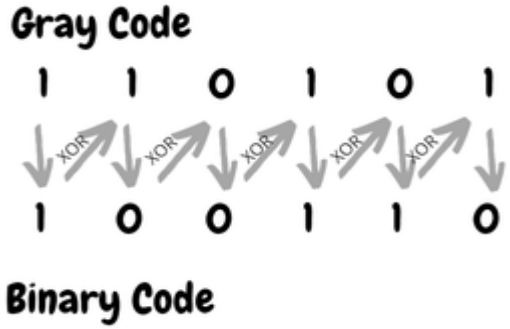
Binary to Gray Conversion:

1. The Most Significant Bit (MSB) of the gray code is always equal to the MSB of the given binary code

2. Other bits of the output gray code can be obtained by XORing binary code bit at that index and previous index

Example: 100110 (Binary) → 110101 (Gray)





Gray to Binary Conversion:

1. The Most Significant Bit (MSB) of the binary code is always equal to the MSB of the given gray code.
2. Other bits of the output binary code can be obtained by checking the gray code bit at that index. If the current gray code bit is 0, then copy the previous binary code bit, else copy the invert of the previous binary code bit.

Example: 110101 (Gray) → 100110 (Binary)

Gray Codes:

A gray code is a list of $'2^n'$ binary sequences of length 'n' such that consecutive sequences in the list differ in exactly one position.

A gray code is called cyclic if the 1st and last sequence in the list also differ in exactly in one position.

Primary Motivation for using the gray codes is the computers. Earlier computers were built using Electro-mechanical switches where the transition from one sequence of 0s and 1s to another sequence of 0s and 1s will happen in a sequential manner.

Ex: 1 and 2 in 3-bit binary are given as 001 and 010. If the transition has to happen 1-bit at a time, 1. 001, 2. 011, and 3. 010.

A n-cube Q_n is the graph whose vertices are labelled by the binary sequences of length 'n' where two vertices are adjacent if and only if their sequences differ in exactly one bit position.

Gray Codes:

1. These are not weighted.

2. Also known as reflected binary code.

3. In modern digital communications, Gray codes play an important role in error correction.

4. **Binary Reflected Gray Code (BRGC).** Ex: **000, 001, 011, 010, 110, 111, 101, 100. BRGC codes are also a Gray code (Proof is part of Discrete Mathematics theory). BRGC (Gray codes) finds their applications in range encoding, graph theory, and so on.**

5. K-Maps uses gray codes (will be elaborated in Unit-2)

Binary Coded Decimal (BCD)

- **The BCD code is the 8,4,2,1 code.**
- **This code is the simplest, most intuitive binary code for decimal digits and uses the same weights as a binary number, but only encodes the first ten values from 0 to 9.**
- **Example: 1001 (9) = 1000 (8) + 0001 (1)**
- ***How many "invalid" code words are there?***
- ***What are the "invalid" code words?***

Excess-3 Code and 8, 4, –2, –1 Code

Decimal	Excess-3	8, 4, –2, –1
0	0011	0000
1	0100	0111
2	0101	0110
3	0110	0101
4	0111	0100
5	1000	1011
6	1001	1010
7	1010	1001
8	1011	1000
9	1100	1111

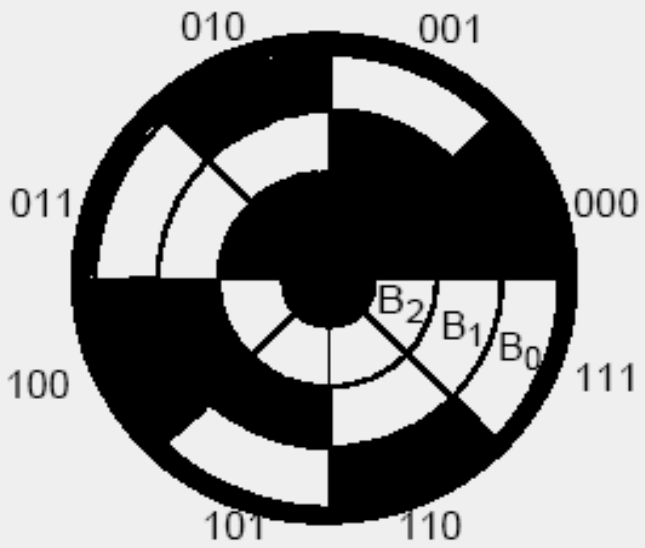
- **What property is common to these codes?**
  - These are reflected codes; complementing is performed simply by replacing 0's by 1's and vice-versa

Gray Code

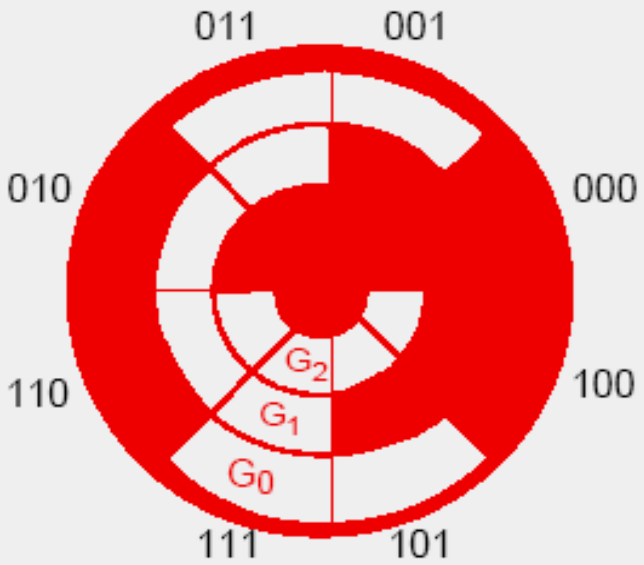
Decimal	8,4,2,1	Gray
0	0000	0000
1	0001	0100
2	0010	0101
3	0011	0111
4	0100	0110
5	0101	0010
6	0110	0011
7	0111	0001
8	1000	1001
9	1001	1000

- **What property does this Gray code have?**
  - Counting up or down changes only one bit at a time (including counting between 9 and 0)

Gray Code: Optical Shaft Encoder



(a) Binary Code for Positions 0 through 7



(b) Gray Code for Positions 0 through 7

- **Shaft encoder: Capture angular position (e.g., compass)**
- **For binary code, what values can be read if the shaft position is at boundary of “3” and “4” (011 and 100) ?**
- **For Gray code, what values can be read ?**

Warning: Conversion or Coding?

- Do <u>NOT</u> mix up <u>conversion</u> of a decimal number to a binary number with <u>coding</u> a decimal number with a BINARY CODE.
- $13_{10} = 1101_2$ (This is <u>conversion</u>)
- 13 $\Leftrightarrow$ 0001|0011 (This is <u>coding</u>)

Binary Arithmetic

- **Single Bit Addition with Carry**
- **Multiple Bit Addition**
- **Single Bit Subtraction with Borrow**
- **Multiple Bit Subtraction**
- **Multiplication**
- **BCD Addition**

Binary Addition

- Binary addition is very simple.
- This is best shown in an example of adding two binary numbers…

```
   1   1   1   1   1   1 <------------ carries
       1   1   1   1   0   1
+          1   0   1   1   1
-----------------------------
   1   0   1   0   1   0   0
```

Binary Subtraction

- We can also perform subtraction (with borrows in place of carries).
- Let's subtract $(10111)_2$ from $(1001101)_2$…

```
          1          10
        0  10  10   0  0  10     ←———— borrows

        1   0   0   1  1   0   1
-               1   0  1   1   1
-----------------------------------
            1   1   0  1   1   0
```

Binary Multiplication

- Binary multiplication is much the same as decimal multiplication, except that the multiplication operations are much simpler…

```
                1   0   1   1   1
X                   1   0   1   0
-----------------------------------
                0   0   0   0   0
            1   0   1   1   1
        0   0   0   0   0
    1   0   1   1   1
-----------------------------------
    1   1   1   0   0   1   1   0
```

Error-Detection Codes

- Redundancy (e.g. extra information), in the form of extra bits, can be incorporated into binary code words to detect and correct errors.
- A simple form of redundancy is parity, an extra bit appended onto the code word to make the number of 1's odd or even. Parity can detect all single-bit errors and some multiple-bit errors.
- A code word has even parity if the number of 1's in the code word is even.
- A code word has odd parity if the number of 1's in the code word is odd.

Code-Word: Information-bits + redundant bits together is referred to as a *code-word*.

The following information "1000001" has to be transmitted from one place to another via a transmission medium such as twisted-pair. Due to some transmission parameters such as white-noise or EMI interference, the data may get corrupted. Data corruption mean one or more bits in message transmitted by the sender may arrive in a different manner/form at the receiver. For eg., the receiver might have received "1100001" instead of "1000001". So transmitting code-words instead of naïve information is the solution.

Parity Codes

- Parity codes are formed by concatenating a *parity bit*, *P* to each code word of C.
- In an *odd-parity code*, the parity bit is specified so that the total number of ones is odd.
- In an even-*parity code*, the parity bit is specified so that the total number of ones is even.

P	Information

1 1 0 0 0 0 1 1

↑

Added even parity bit

0 1 0 0 0 0 1 1

↑

Added odd parity

3-Bit Parity Code Example

- **Fill in the even and odd parity bits:**

Even Parity	Odd Parity
Message-Parity	Message-Parity
000_	000_
001_	001_
010_	010_
011_	011_
100_	100_
101_	101_
110_	110_
111_	111_

- **The binary codeword "1111" has <u>even parity</u> and the binary code "1110" has <u>odd parity</u>. Both could be used to represent data.**

ASCII Code

- American Standard Code for Information Interchange
- ASCII is a 7-bit code, frequently used with an 8th bit for error detection.
- A – Z (26 codes), a – z (26 codes), 0-9 (10 codes), others (@#$%^&*….)

Character	ASCII (bin)	ASCII (hex)	Decimal	Octal
A	1000001	41	65	101
B	1000010	42	66	102
C	1000011	43	67	103
…				
Z	1011010	5A	90	132
a	1100001	61	97	141
…				
1	0110001	31	49	061
;	0111011	3B	59	073

ASCII Character Codes

- **American Standard Code for Information Interchange**
- **This code is the most popular code used to represent information sent as character-based data. It uses 7-bits to represent:**
  - 94 Graphic printing characters.
  - 34 Non-printing characters
- Some non-printing characters are used for text format (e.g. BS = Backspace, CR = carriage return)
- Other non-printing characters are used for record marking and flow control (e.g. STX and ETX start and end text areas).
- ASCII is a 7-bit code, but most computers manipulate 8-bit quantity called *byte.* To detect errors, the 8th bit is used as a parity bit.

Signed Numbers

- Plus and minus sign used for decimal numbers: 25 (or +25), -16, etc.
- For computers, desirable to represent everything as ***bits***.
- Three types of signed binary number representations: signed magnitude, 1's complement, 2's complement.
- In each case: left-most bit indicates sign: positive (0) or negative (1).

$$\underline{\mathbf{0}\,\mathbf{0001100}_2} = \mathbf{12}_{10}$$

Sign bit **Magnitude**

$$\underline{\mathbf{1}\,\mathbf{0001100}_2} = \mathbf{-12}_{10}$$

Sign bit **Magnitude**

Signed Magnitude Representation

Signed Magnitude Representation

- MSB is used as sign bit. Remaining bits barring MSB will represent the magnitude
- Range: $-2^{n-1} - 1 \; \ldots\ldots \; 0 \; \ldots\ldots + 2^{n-1} - 1$ (n is the number of bits)
- Issue: +0 (0000) and -0 (1000) binary representations are different
- Number Representation for 4-bit system: 0 (0000), 1 (0001), 2 (0010), 3 (0011), 4 (0100), 5 (0101), 6 (0110), 7 (0111), -0 (1000), -1 (1001), -2 (1010), -3, -4, -5, -6, -7 (give the binary rep. for the leftover neg. no.)

$\mathbf{0\underline{0001100}_2 = 12_{10}}$

Sign bit **Magnitude**

$\mathbf{1\underline{0001100}_2 = -12_{10}}$

Sign bit **Magnitude**

1's Complement

- The one's complement of a binary number involves inverting all bits (individually).
  - 1's comp of 00110011 is 11001100
  - 1's comp of 10101010 is 01010101
- For an n bit number 'N', the 1's complement is $(2^n-1) - N$.
- Called diminished radix complement since 1's complement for base (radix 2).
- To find negative of a number, take the 1's complement of that number (provided if we are using 1's complement number system).

$\mathbf{0\underline{0001100}_2 = 12_{10}}$

Sign bit **Magnitude**

$\mathbf{1\underline{1110011}_2 = -12_{10}}$

Sign bit **Magnitude**

1's Complement

- Range: $-2^{n-1} - 1 \ \ldots\ldots.0 \ \ldots\ldots + 2^{n-1} - 1$ (n is the number of bits)
- Issue: +0 (0000) and -0 (1000) binary representations are different
- Number Representation for 4-bit system: 0 (0000), 1 (0001), 2 (0010), 3 (0011), 4 (0100), 5 (0101), 6 (0110), 7 (0111), -0 (1111), -1 (1110), -2 (1101), -3, -4, -5, -6, -7 (give the binary rep. for the leftover neg. no.)

2's Complement

- The two's complement of a binary number involves inverting all bits and adding 1 at the LSB position.
  - 2's comp of 00110011 is 11001101
  - 2's comp of 10101010 is 01010110
- For an n bit number N the 2's complement is $(2^n-1) – N + 1$.
- Called radix complement since 2's complement for base (radix 2).
- To find negative of a number, take the 2's complement (provided if we are using 1's complement number system)..

2's Complement

- Range: -2^{n-1} ... ... 0 ... ... $+2^{n-1}-1$ (n is the number of bits)
- **Adv:** +0 (0000) and -0 (0000) has the same representation.
- Number Representation for 4-bit system: 0 (0000), 1 (0001), 2 (0010), 3 (0011), 4 (0100), 5 (0101), 6 (0110), **7** (0111), -0 (0000), -1 (1111), -2 (1110), -3, -4, -5, -6, -7, **-8** (give the binary rep. for the leftover neg. no.)

$\mathbf{00001100_2 = 12_{10}}$

Sign bit **Magnitude**

$\mathbf{11110100_2 = -12_{10}}$

Sign bit **Magnitude**

2's Complement Shortcuts

- **Algorithm 1** – Simply complement each bit and then add 1 to the result.

  Finding the 2's complement of $(01100101)_2$ and of its 2's complement…

```
N = 01100101                [N] = 10011011
    10011010                    01100100
  +        1                  +        1
    ---------------             ---------------
    10011011                    01100101
```

- **Algorithm 2** – Starting with the least significant bit, copy all of the bits up to and including the first 1 bit and then complementing the remaining bits.

```
N = 0 1 1 0 0 1 0 1          N = 00110000
[N] = 1 0 0 1 1 0 1 1        N = 11010000
```

Signed, 1's and 2's Complement – 4-bit System

Number-Decimal	Signed Magnitude	1's Complement	2's Complement
-8			1000
-7	1111	1000	1001
-6	1110	1001	1010
-5	1101	1010	1011
-4	1100	1011	1100
-3	1011	1100	1101
-2	1010	1101	1110
-1	1001	1110	1111
0 (-0)	**00000 (1000)**	**0000 (1111)**	***0000***
+1	0001	0001	0001
+2	0010	0010	0010
+3	0011	0011	0011
+4	0100	0100	0100
+5	0101	0101	0101
+6	0110	0110	0110
+7	0111	0111	0111

ARITHMETIC: 1's Complement

Addition:

- Using 1's complement numbers, adding numbers is easy.
- For example, suppose we wish to add $+(1100)_2$ and $+(0001)_2$.
- Let's compute $(12)_{10} + (1)_{10}$.

  $(12)_{10} = +(1100)_2 = 01100_2$ in 1's comp.

  $(1)_{10} = +(0001)_2 = 00001_2$ in 1's comp.

Step 1: Add binary numbers

Step 2: Add carry to low-order bit

```
                0 1 1 0 0
+               0 0 0 0 1
-----------------------
            0 0 1 1 0 1
Add carry   └──────────→ 0
-----------------------
Final
Result          0 1 1 0 1
```

ARITHMETIC: 1's Complement

Subtraction:

- Using 1's complement numbers, subtracting numbers is also easy.
- For example, suppose we wish to subtract $+(0001)_2$ from $+(1100)_2$.
- Let's compute $(12)_{10}$ - $(1)_{10}$.
  - $(12)_{10} = +(1100)_2 = 01100_2$ in 1's comp.
  - $(-1)_{10} = -(0001)_2 = 11110_2$ in 1's comp.

Step 1: Take 1's complement of 2nd operand
Step 2: Add binary numbers
Step 3: Add carry to low order bit

```
                 0 1 1 0 0
              -  0 0 0 0 1
                 -----------
1's comp
Add
                 0 1 1 0 0
              +  1 1 1 1 0
Add carry        -----------
                 1 0 1 0 1 0
                           1
Final            -----------
Result           0 1 0 1 1
```

ARITHMETIC: 2's Complement

Addition:

- Using 2's complement numbers, adding numbers is easy.
- For example, suppose we wish to add $+(1100)_2$ and $+(0001)_2$.
- Let's compute $(12)_{10} + (1)_{10}$.
  - $(12)_{10} = +(1100)_2 = 01100_2$ in 2's comp.
  - $(1)_{10} = +(0001)_2 = 00001_2$ in 2's comp.

Step 1: Add binary numbers
Step 2: Ignore carry bit

		0	1	1	0	0
Add	+	0	0	0	0	1
Final Result		0	1	1	0	1

Ignore

ARITHMETIC: 2's Complement

Subtraction:

- For example, suppose we wish to subtract $+(0001)_2$ from $+(1100)_2$.
- Let's compute $(12)_{10}$ - $(1)_{10}$.

  $(12)_{10} = +(1100)_2 = 01100_2$ in 2's comp.

  $(-1)_{10} = -(0001)_2 = 11111_2$ in 2's comp.

Step 1: Take 2's complement of 2nd operand

Step 2: Add binary numbers

Step 3: Ignore carry bit

```
                 0 1 1 0 0
              -  0 0 0 0 1
            ----------------------
```

Question-1: What difference did you observe between 1's comp. and 2's comp. arithmetic.?

Question-2: Which should/can be preferred.?

```
2's comp
                 0 1 1 0 0
Add           +  1 1 1 1 1
            ----------------------
Final
Result           1 0 1 0 1 1
                 ↑
                 Ignore
```

ARITHMETIC: 2's Complement

Subtraction:

Example-2:

- Let's compute $(13)_{10} - (5)_{10}$.

  $(13)_{10} = +(1101)_2 = (01101)_2$

  $(-5)_{10} = -(0101)_2 = (11011)_2$

- Adding these two 5-bit codes…

```
                          0 1 1 0 1
carry                +    1 1 0 1 1
                     --------------
                  1       0 1 0 0 0
```

- Discarding the carry bit, the sign bit is seen to be zero, indicating a correct result. Indeed, $(01000)_2 = +(1000)_2 = +(8)_{10}$.

ARITHMETIC: 2's Complement

Subtraction:

Example-3:

- Let's compute $(5)_{10} - (12)_{10}$.

  $(-12)_{10} = -(1100)_2 = (10100)_2$

  $(5)_{10} = +(0101)_2 = (00101)_2$

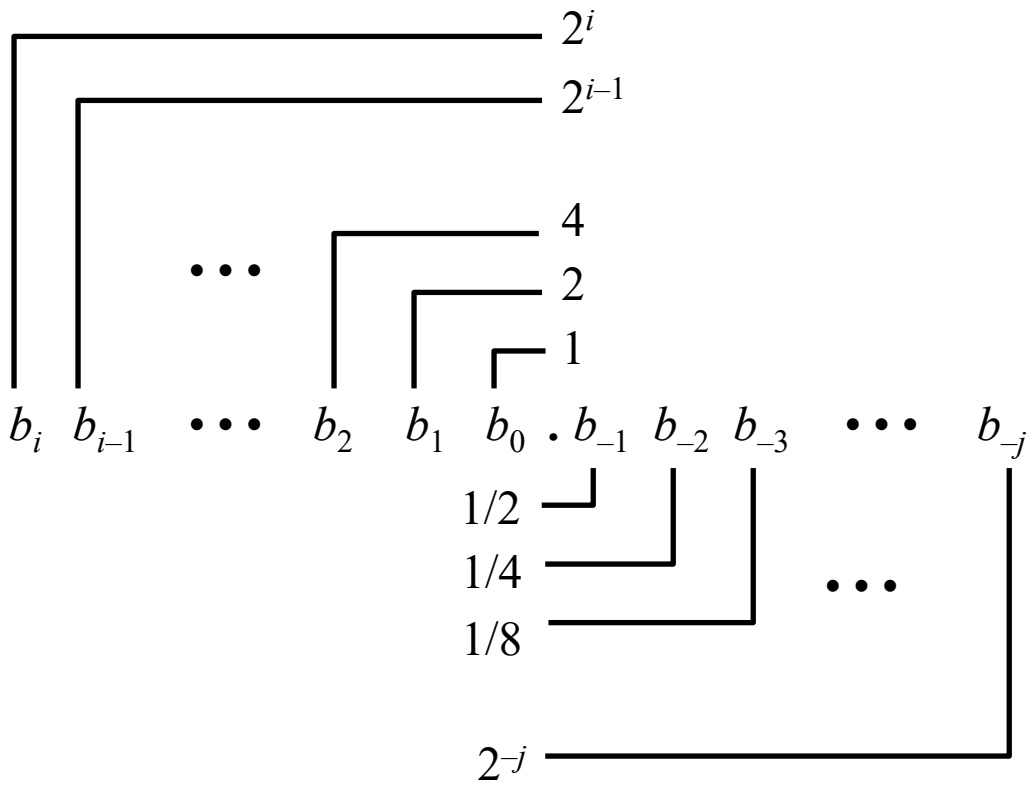
- Adding these two 5-bit codes…

```
      0 0 1 0 1
+     1 0 1 0 0
  --------------
      1 1 0 0 1
```

- Here, there is no carry bit and the sign bit is 1. This indicates a negative result, which is what we expect. $(11001)_2 = -(7)_{10}$.

Floating Point Numbers: IEEE 754 Standard

Fractional Binary Numbers



- Representation
  - Bits to right of "binary point" represent fractional powers of 2
  - Represents rational number:

$$\sum_{k=-j}^{i} b_k \cdot 2^k$$

Fractional Binary Numbers

Value	Representation
5-3/4	101.11_2
2-7/8	10.111_2
63/64	0.111111_2

Observation

- Divide by 2 by shifting right
- Numbers of form $0.111111\ldots_2$ just below 1.0
  - Use notation $1.0 - \varepsilon$

Limitation

- Can only exactly represent numbers of the form $x/2^k$
- Other numbers have repeating bit representations

Value	Representation
1/3	$0.0101010101[01]\ldots_2$
1/5	$0.001100110011[0011]\ldots_2$
1/10	$0.0001100110011[0011]\ldots_2$

Floating Point Representation

Numerical Form

$-1^{s}\ M\ \ 2^{E}$

Sign bit s determines whether number is negative or positive

Significand M normally a fractional value in range [1.0,2.0).

Exponent E weights value by power of two

Encoding

MSB is sign bit

`exp` field encodes E

`frac` field encodes M

Sizes

Single precision: 8 `exp` bits, 23 `frac` bits
(32 bits total)

Double precision: 11 `exp` bits, 52 `frac` bits
(64 bits total)

“Normalized” Numeric Values

- Condition
  - `exp` ≠ `000…0` and `exp` ≠ `111…1`

- Exponent coded as *biased* value
  - *E* = *Exp* – *Bias*
    - *Exp* : unsigned value denoted by **`exp`**
    - *Bias* : Bias value
      - Single precision: 127 (*Exp*: 1…254, *E*: -126…127)
      - Double precision: 1023 (*Exp*: 1…2046, *E*: -1022…1023
      - in general: *Bias* = 2^{m-1} - 1, where m is the number of exponent bits

- Significand coded with implied leading 1
  - *m* = `1.xxx…x`$_2$
    - **`xxx…x`**: bits of `frac`
    - **Get extra leading bit for “free”**

Example

Value

```
Float F = 15213.0;
```

$15213_{10} = 11101101101101_2 \times 2^0 = 1.1101101101101_2 \times 2^{13}$

Significand

$M = 1.\underline{1101101101101}_2$

$\text{Frac} = \underline{1101101101101}0000000000_2$

Exponent

$E = 13$

$Bias = 127$

$Exp = 140 = 10001100_2$

15213 in IEEE 754 Single precision:

0	**10001100**	**11011011011010000000000**
S	**Exponent**	**Mantissa**

Example

Value

```
Float F = -150.75;
```

$(150.75)_{10} = 10010110.11_2 \times 2^0 \quad = 1.001011011_2 \times 2^7$

Significand

$M = \quad 1.001011011_2$

Frac = $00101101100000000000000_2$

Exponent

$E = 7$

$Bias = 127$

$Exp = 134 \quad = \quad 10000110_2$

15213 in IEEE 754 Single precision:

1	10000110	00101101100000000000000
S	Exponent	Mantissa

Special Cases

`Cases:`

Condition

- exp = 000…..0

Value

- Exponent value = -bias + 1
- Significand value m = $0.XXXXXXXX_2$
  - XXXXXXXX: bits of fraction.

1. `exp` = `000…0`, `frac` = `000…0`
   - Represents value 0
   - Note that have distinct values +0 and –0

2. `exp` = `000…0`, `frac` ≠ `000…0`
   - Numbers very close to 0.0
   - Lose precision as get smaller
   - “Gradual underflow”

Special Cases

Condition

`exp` = `111…1`

Cases

3. `exp` = `111…1`, `frac` = `000…0`

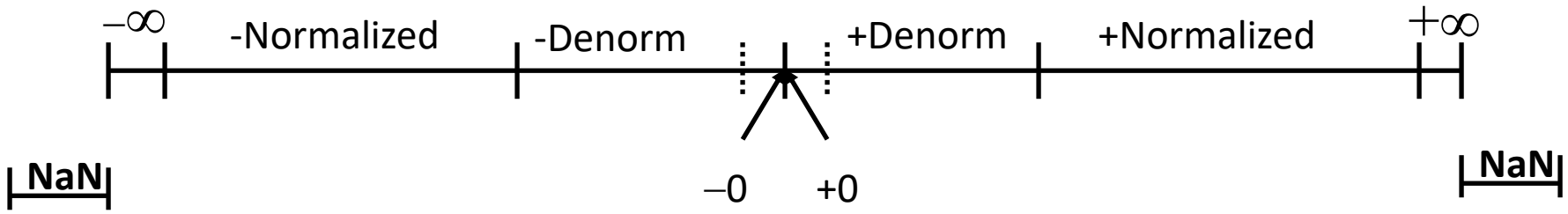
   Represents value ∞ (infinity)
   Operation that overflows
   Both positive and negative

   E.g., 1.0/0.0 = –1.0/–0.0 = +∞, 1.0/–0.0 = –∞

4. `exp` = `111…1`, `frac` $\neq$ `000…0`

   Not-a-Number (NaN)
   Represents case when no numeric value can be determined
   E.g., sqrt(–1), $\infty - \infty$

Summary



Summary of Special Cases

			Single Precision		Double Precision	
	Sign	Fraction	Biased Exponent	Value	Biased Exponent	Value
Positive Zero	0	0	0	0	0	0
Negative Zero	1	0	0	0	0	0
Plus Infinity	0	0	255 (all 1's)	$+\infty$	2047 (all 1's)	$+\infty$
Minus Infinity	1	0	255 (all 1's)	$-\infty$	2047 (all 1's)	$-\infty$
Not a Number	0 or 1	$\neq 0$	255 (all 1's)	NaN	2047 (all 1's)	NaN
Positive normalized nonzero	0	M	$0 < E' < 255$	$2^{E'-127}$ (1.M)	$0 < E' < 2047$	$2^{E'-1023}$ (1.M)
Negative normalized nonzero	1	M	$0 < E' < 255$	$-2^{E'-127}$ (1.M)	$0 < E' < 2047$	$-2^{E'-1023}$ (1.M)
Positive denormalized	0	$M \neq 0$	0	$-2^{E'-127}$ (0.M)	0	$-2^{E'-1023}$ (0.M)
Negative denormalized	1	$M \neq 0$	0	$-2^{E'-127}$ (0.M)	0	$-2^{E'-1023}$ (0.M)

Floating Point Arithmetic

Floating Point Add/ Subtract

- Check for zeros
- Align the mantissa
- Add or subtract mantissas
- Normalize the result

Example:

1. Add the numbers 3.56 and 11.25 as per IEEE 754.
2. Subtract 3.56 from 11.25 as per IEEE 754.

Floating Point Arithmetic

Floating Point Multiplication

- Check for zeros
- Add the exponents
- Multiply the mantissas
- Normalize the product

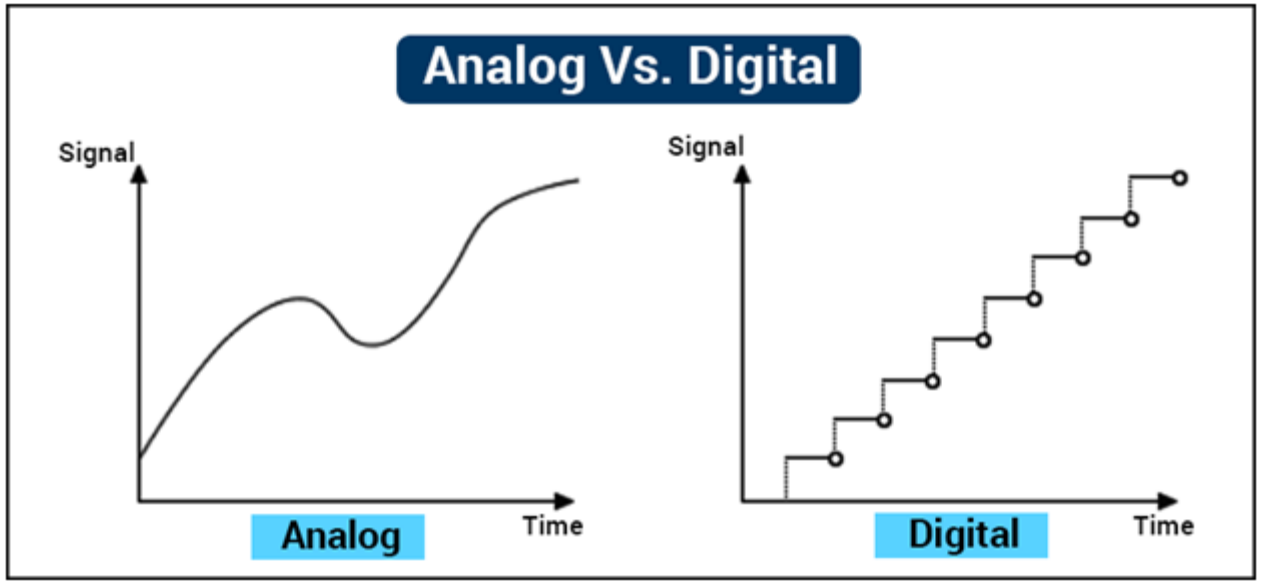
Example:

1. Multiply the numbers 3.56 and 11.25 as per IEEE 754.

Digital Circuits

Digital Circuits

- Digital Circuits and Digital Systems operate on digital data.
- It is important to understand the difference between analog and digital data (refer to the image below).
- Digital systems/circuits usually operate on digital data which is in the binary form.



Digital Circuits

Why Binary System.?

- If we use decimal number system for building digital systems, then we need to generate, manipulate, and store 10 different logic levels. This will make the system design more complex.
- It is relatively easier to build circuits which can handle only 2 logic levels
- Therefore binary number systems are used in the designing of digital systems
- It is good to recall your **ASCII** encoding once.

Digital Circuits

- The major components and the basic building block of any given digital circuit are logic gates.

  Ex: AND, OR, NOT, NAND, etc

- These circuits operate only on two states: 1. High (+5V) and 2. Low (0V). They take inputs in these two states, and process them again in these two states and finally provide the output in these two states alone.

- They consume less power compared to the analog circuits

- Digital circuits cannot interact with the real-world data directly.

  - Real world data exists only in analog form

  - Need to be converted into digital form before given as an input to the digital system or circuit for processing.

  - Note, analog circuits will work based on the analog inputs directly

  - Examples of converters: ADC, DAC, etc.

Digital Circuits

- Digital circuits compared to their analog counterparts, less sensitive to the changes in the input.

  Ex: refer to the image below

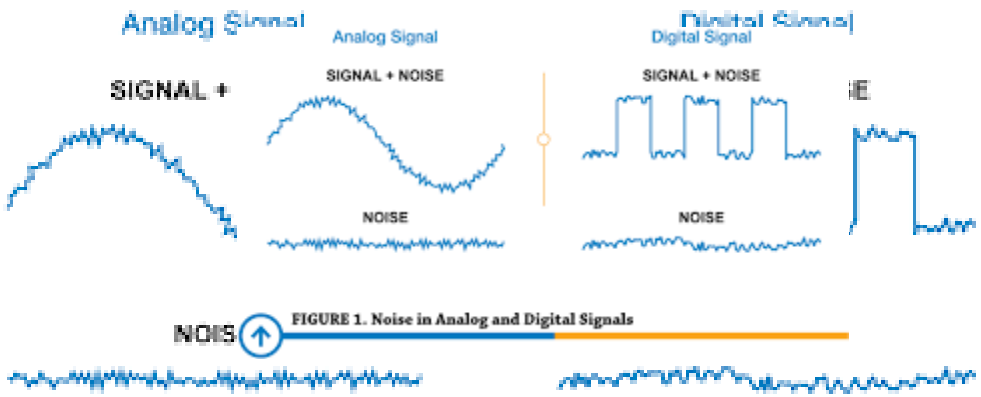


FIGURE 1. Noise in Analog and Digital Signals

- Digital circuits are also more immune to noise than their analog counterparts.

Digital Circuits

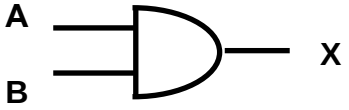
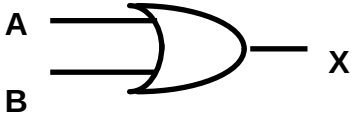
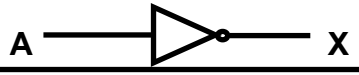
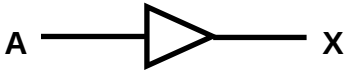
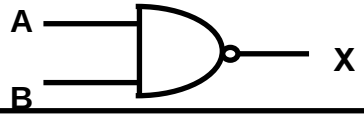
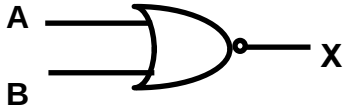
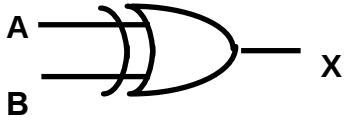
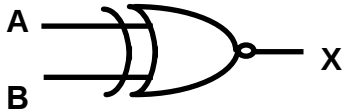
Accuracy: They are more accurate than analog circuits

Triggering: Digital circuits rely on clock signal for synchronization purposes

Applications:

Analog Circuits	Digital Circuits
Data Communication, Radio Stations, Power Transmission etc.	Computers, Calculators, Microcontrollers etc.

Digital Circuits- Logic Gates

Name	Symbol	Function	Truth Table
AND		X = A • B or X = AB	A B X 0 0 0 0 1 0 1 0 0 1 1 1
OR		X = A + B	A B X 0 0 0 0 1 1 1 0 1 1 1 1
I		X = A’	A X 0 1 1 0
Buffer		X = A	A X 0 0 1 1
NAND		X = (AB)’	A B X 0 0 1 0 1 1 1 0 1 1 1 0
NOR		X = (A + B)’	A B X 0 0 1 0 1 0 1 0 0 1 1 0
XOR Exclusive OR		X = A ⊕ B or X = A’B + AB’	A B X 0 0 0 0 1 1 1 0 1 1 1 0
XNOR Exclusive NOR or Equivalence		X = (A ⊕ B)’ or X = A’B’+ AB	A B X 0 0 1 0 1 0 1 0 0 1 1 1

Thank You